

Agentic AI Systems: A Conceptual Overview (Demo)

From Single-Turn Generation to Agents

An 'agentic' AI system extends a language model beyond single-turn text generation by giving it the ability to take actions — calling tools, querying external systems, or invoking other models — and to use the results of those actions to decide what to do next, typically in a loop until some stopping condition is met.

This turns the model from a passive text generator into something closer to a controller that plans, acts, observes results, and revises its plan, which introduces new failure modes that a single-turn system does not have, such as infinite loops, runaway tool calls, or cascading errors from one bad intermediate step.

Common Agent Roles

Multi-agent systems often decompose responsibility into specialized roles: a planner that breaks a high-level request into sub-tasks, a worker or research agent that executes sub-tasks (for example, retrieving information from a knowledge base), a critic that checks the worker's output against some quality bar, and a reporting agent that assembles a final structured output once the critic is satisfied.

Separating these responsibilities makes each agent's job narrower and easier to validate individually, and makes it possible to insert a review or revision step between generation and finalization rather than trusting a single pass end-to-end.

Tool Use and MCP

Agents commonly interact with the outside world through defined 'tools' — functions with a name, a description, and a validated input/output schema — that the model can choose to call based on the current task. The Model Context Protocol (MCP) formalizes this as a client-server interface: an MCP server exposes a set of tools with schemas, and an MCP client (embedded in the agent) discovers and invokes those tools over a standard protocol, rather than the tools being hardcoded Python functions called directly by the agent's own process.

This separation matters because it allows tools to be swapped, reused across different agents, and audited independently of the agent's internal reasoning code.

Human-in-the-Loop Control

Because agentic systems can take consequential actions autonomously, many designs require human approval at specific checkpoints: before publishing a final result, when confidence falls below a threshold, when evidence sources contradict each other, or after a maximum number of revision cycles has been reached without the critic approving the output. Supporting resume-from-pause — where the workflow state is persisted so the system can wait indefinitely for a human decision and then continue exactly where it left off — is a common requirement for this kind of control.

Reliability: Loops, Timeouts, and Recovery

Because agent loops can in principle run indefinitely, production designs enforce explicit maximum iteration counts, timeouts on individual tool calls, and a defined retry policy for transient tool failures. When a tool call fails or returns a malformed response, the system should recover gracefully — for example, by retrying with backoff or surfacing the failure to a human — rather than crashing or silently proceeding as if the call had succeeded.